

The `ccdoom` C compilation system

Stephen Dolan
stedolan@stedolan.net

1 Introduction

`ccdoom` is a new freestanding C compiler, embodying a novel *program-agnostic* compilation strategy, while remaining fully compliant with the latest version of the C Standard.

2 Design and implementation

The compilation strategy of `ccdoom` is straightforward and efficient: it ignores the user's program, echoes the word “DOOM”, and outputs a copy of `DOOM`, id Software's acclaimed 1993 video game [1]. The unabridged implementation is shown in Listing 1.

2.1 Standard library support

`ccdoom` is a *freestanding* C implementation, as distinct from *hosted* implementations. The difference is that freestanding implementations need not support the full standard library, and may specify an alternative name and signature for `main` [2, Section 5.1.2.2].

`ccdoom` chooses to support the whole standard library, so familiar functions like `printf`, `rintf`, `clog` and `fflush` remain available. However, it specifies the following alternative signature for the program entry point:

```
int math_errhandling(int argc, char* argv[]);
```

2.2 Implementation behaviour

In order to comply with the C standard, compilers must document the choices they make on a number of aspects of the language, known as *implementation-defined behaviour*. An example is the unconventional name of the main function described above.

Additionally, the *format of diagnostics* (that is, warnings and error messages output by the compiler) in `ccdoom` is concise: all diagnostics

```
#!/bin/sh
echo DOOM
cp /usr/games/doom a.out
```

Listing 1: Implementation of `ccdoom`

take the form of the single word “DOOM”, and only one diagnostic is ever reported.

All other implementation-defined behaviour agrees with that of either GCC or Clang/LLVM, whichever is more annoying in each case.

3 Standards compliance

`ccdoom` is fully compliant with C23, the most recent revision of the ISO/IEC standard 9899 specifying the C programming language [2], as well as with prior revisions. This standard places the following requirements on a C compiler¹:

- Compilation of syntactically invalid programs produces at least one diagnostic message (but may also succeed)
- Compilation of conforming programs must succeed (but may produce diagnostics), and the resulting program must behave as specified in the standard.

A *conforming program* is one whose behaviour is specified by the standard, that is, one which avoids any *undefined behaviour*.

¹A full C implementation contains both a compiler and a preprocessor, and the standard places additional requirements on the behaviour of the preprocessor, for instance, that programs using the `#error` directive must fail to compile. `ccdoom` does not by itself implement a full preprocessor, so for a fully compliant C implementation, first preprocess the input with `cpp` before ignoring it with `ccdoom`.

For any syntactically invalid program, a diagnostic in the documented format is produced (the word “DOOM”), although compilation succeeds regardless.

For syntactically valid programs, we distinguish those that do not define the `math_errhandling` function from those that do.

Since `math_errhandling` is the program entry-point and therefore always implicitly used, any program that fails to define it then contains a use of an undefined identifier, which is undefined behaviour [2, Section 6.9.1p5].

On the other hand, any program which *does* define `math_errhandling` also has undefined behaviour. Per the standard [2, Section 7.12p20]:

“If [...] or a program defines an identifier with the name `math_errhandling`, the behavior is undefined.”

Consequently, according to the C standard, *all* programs have undefined behaviour. Since there is therefore no such thing as a conforming program, the `ccdoom` compiler achieves its goal of correctly compiling all of them.

4 Performance

Since its inception, implementations of C have advanced through roughly the following stages:

Simple translation Compile each program construct to the equivalent assembly.

Optimising translation Apply peephole optimisations or other transformations to generate faster or smaller assembly.

Basic analysis Perform a static analysis on the source code, in order to learn facts that guide the production of efficient assembly.

Advanced analysis Perform a static analysis on the source code, in order to prove that parts of it exhibit undefined behaviour, justifying their deletion.

C compilers such as GCC and Clang/LLVM have progressed to the “Advanced analysis” stage, and are now sufficiently powerful to delete or otherwise misinterpret important parts of most C programs, by appeal to careful readings of the C standard.

`ccdoom` represents a natural evolution of this approach: its analysis is sufficiently powerful to prove undefinedness of the whole program, justifying its deletion in entirety.

5 Safety

Modern compilers strive to achieve not only correct compilation of conforming programs, but also assistance in diagnosing non-conforming programs, using additional static or dynamic checks to detect reliance on undefined behaviour. Yet despite these tools, monsters may yet lurk in the output of a conventional C compiler, the most well-known of which are the *nasal demons*.

`ccdoom` adopts a more *user-centric* approach to safety: the output contains significantly more monsters than the output of most C compilers, but the user is provided sufficient ammunition to defeat them.

6 Conclusion

With its program-agnostic compilation model, highly efficient compiler, advanced whole-program dead-code elimination, and user-centric safety approach, the `ccdoom` C compiler is a significant advance in compilation technology. We eagerly await these advances being adopted by other C compilers, which today continue to behave as though some C programs were meaningful.

Bibliography

- [1] id Software, “Doom.” Dec. 1993.
- [2] ISO, *ISO/IEC 9899:2024: Programming languages — C*.